

```
from google.colab import files
uploaded = files.upload()
```



Choose files CoolingTimeData.csv

- **CoolingTimeData.csv**(text/csv) - 54822 bytes, last modified: 29/04/2025 - 100% done

Saving CoolingTimeData.csv to CoolingTimeData.csv

```
import pandas as pd
```

```
# Try using a different encoding
```

```
df = pd.read_csv("CoolingTimeData.csv", encoding='ISO-8859-1')
```

```
# Preview the dataset
```

```
print("\n First 5 rows:")
```

```
print(df.head())
```

```
# Column info
```

```
print("\n Column info:")
```

```
print(df.info())
```

```
# Missing values
```

```
print("\n ! Missing values in each column:")
```

```
print(df.isnull().sum())
```



First 5 rows:

	Mold material	Product thickness (mm)	Coolant temp (°C)	\
0	Steel	2.0	20	
1	Steel	2.5	18	
2	Aluminum	3.0	16	
3	Aluminum	3.5	14	
4	Copper	4.0	12	

	Injection pressure (bar)	Ambient temp (°C)	Cycle time (s)	Plastic type	\
0	500	25	30	ABS	
1	520	26	32	ABS	
2	540	27	34	PP	
3	560	28	36	PP	
4	580	29	38	PC	

	Wall thickness (mm)	Mold temp (°C)	Cavity size (cm³)	Cooling time (s)
0	1.0	80	100	5
1	1.2	85	110	6
2	1.4	90	120	7
3	1.6	95	130	8
4	1.8	100	140	9

Column info:

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 1000 entries, 0 to 999
```

```
Data columns (total 11 columns):
```

#	Column	Non-Null Count	Dtype
0	Mold material	1000 non-null	object
1	Product thickness (mm)	1000 non-null	float64
2	Coolant temp (°C)	1000 non-null	int64
3	Injection pressure (bar)	1000 non-null	int64
4	Ambient temp (°C)	1000 non-null	int64
5	Cycle time (s)	1000 non-null	int64
6	Plastic type	1000 non-null	object
7	Wall thickness (mm)	1000 non-null	float64
8	Mold temp (°C)	1000 non-null	int64
9	Cavity size (cm³)	1000 non-null	int64
10	Cooling time (s)	1000 non-null	int64

```
dtypes: float64(2), int64(7), object(2)
```

```
memory usage: 86.1+ KB
```

```
None
```

! Missing values in each column:

Mold material	0
Product thickness (mm)	0
Coolant temp (°C)	0
Injection pressure (bar)	0
Ambient temp (°C)	0
Cycle time (s)	0
Plastic type	0
Wall thickness (mm)	0
Mold temp (°C)	0
Cavity size (cm³)	0
Cooling time (s)	0

```
dtype: int64
```

```
# Clean column names: remove leading/trailing spaces
```

```
df.columns = [col.strip() for col in df.columns]
```

```

print("✅ Cleaned Columns:")
print(df.columns)

# Encode categorical columns if they exist
cat_cols = df.select_dtypes(include=['object', 'category']).columns
for col in cat_cols:
    df[col] = df[col].astype('category').cat.codes
    print(f"🔢 Encoded column: {col}")

# Drop rows with missing values (simplest option)
df.dropna(inplace=True)

# Confirm dataset shape after cleaning
print(f"\n✅ Dataset shape after cleaning: {df.shape}")

```

```

🔄 ✅ Cleaned Columns:
Index(['Mold material', 'Product thickness (mm)', 'Coolant temp (°C)',
      'Injection pressure (bar)', 'Ambient temp (°C)', 'Cycle time (s)',
      'Plastic type', 'Wall thickness (mm)', 'Mold temp (°C)',
      'Cavity size (cm³)', 'Cooling time (s)'],
      dtype='object')
🔢 Encoded column: Mold material
🔢 Encoded column: Plastic type

✅ Dataset shape after cleaning: (1000, 11)

```

```

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression, Ridge, Lasso, ElasticNet
from sklearn.metrics import mean_squared_error, r2_score

# Target variable: Cooling time (update if your column name is different)
target = "Cooling time (s)"

# Features (X) and target (y)
X = df.drop(columns=[target]) # Exclude target column
y = df[target] # Target variable

# Split the data into training and testing sets (80% training, 20% testing)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Define the models
models = {
    "Linear Regression": LinearRegression(),
    "Ridge Regression": Ridge(alpha=1.0),
    "Lasso Regression": Lasso(alpha=0.1),
    "ElasticNet Regression": ElasticNet(alpha=0.1, l1_ratio=0.5)
}

# Train and evaluate each model
for name, model in models.items():
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    print(f"\n💡 {name}")
    print("R² Score:", r2_score(y_test, y_pred)) # R² score
    print("MSE:", mean_squared_error(y_test, y_pred)) # Mean Squared Error

```

```

🔄 💡 Linear Regression
R² Score: 1.0
MSE: 1.4490301978078886e-26

💡 Ridge Regression
R² Score: 1.0
MSE: 6.422890190634444e-17

💡 Lasso Regression
R² Score: 0.99999999999999857
MSE: 1.1678431905706648e-09

💡 ElasticNet Regression
R² Score: 0.9999999999999964
MSE: 2.919608638970565e-10

```

```

import matplotlib.pyplot as plt
import seaborn as sns

# Create a subplot for each model to show Actual vs Predicted Cooling Time
plt.figure(figsize=(12, 10))
plt.suptitle("Actual vs Predicted Cooling Time", fontsize=16)

# Loop through each model to plot actual vs predicted values
for i, (name, model) in enumerate(models.items(), 1):

```

```

for i, (name, model) in enumerate(models.items()):
    y_pred = model.predict(X_test)

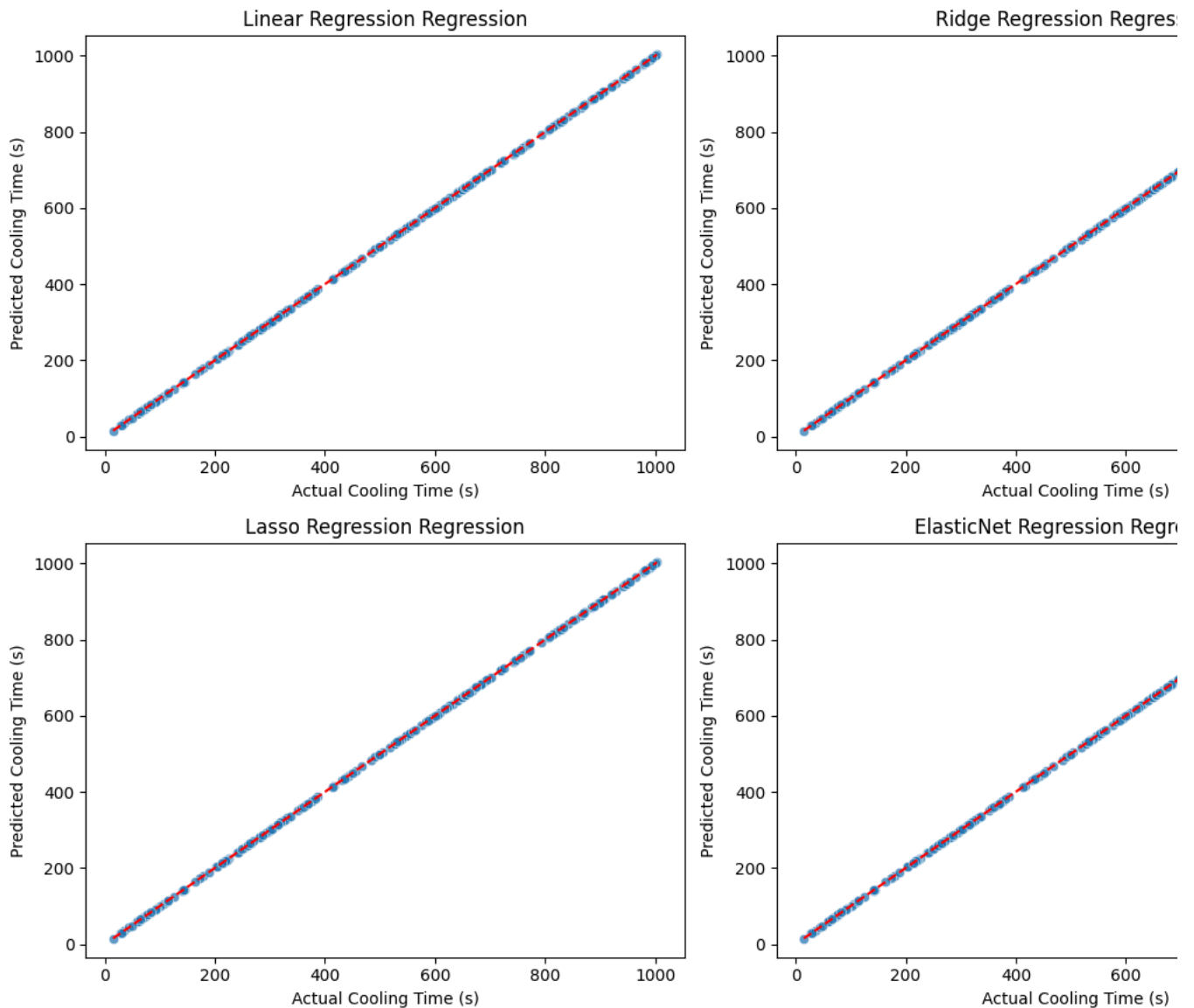
    plt.subplot(2, 2, i) # 2x2 grid of subplots
    sns.scatterplot(x=y_test, y=y_pred, alpha=0.7) # Scatter plot for Actual vs Predicted
    plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--') # Diagonal line (ideal prediction)
    plt.xlabel("Actual Cooling Time (s)")
    plt.ylabel("Predicted Cooling Time (s)")
    plt.title(f"{name} Regression")

plt.tight_layout(rect=[0, 0.03, 1, 0.95]) # Adjust spacing
plt.show()

```



Actual vs Predicted Cooling Time



```

# Choose the best model (for feature importance, let's use Lasso Regression)
best_model = models["Lasso Regression"] # Or you can choose any other model here

# Get the model's coefficients (importance)
importance = best_model.coef_
features = X.columns

# Create a DataFrame to display feature importance
importance_df = pd.DataFrame({
    'Feature': features,
    'Importance': importance
}).sort_values(by='Importance', key=abs, ascending=False)

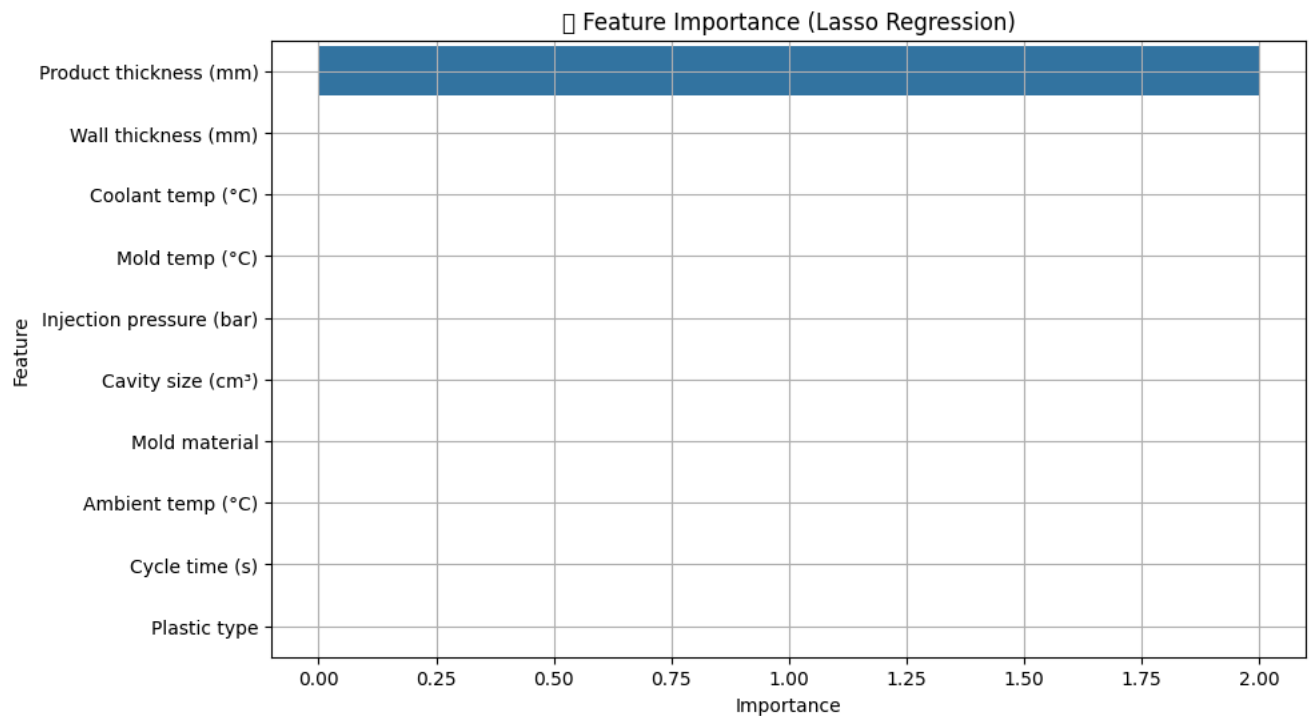
# Plot the feature importance
plt.figure(figsize=(10, 6))
sns.barplot(x='Importance', y='Feature', data=importance_df)

```

```
plt.title("🔍 Feature Importance (Lasso Regression)")
plt.grid(True)
plt.show()
```

```
# Show top features
print("✅ Top features based on Lasso Regression:")
print(importance_df.head())
```

🔗 /usr/local/lib/python3.11/dist-packages/IPython/core/pylabtools.py:151: UserWarning: Glyph 128269 (\N{LEFT-POINTING MAGNIFYING GLASS}) not found in the font. fig.canvas.print_figure(bytes_io, **kw)



✅ Top features based on Lasso Regression:

	Feature	Importance
1	Product thickness (mm)	1.999873e+00
7	Wall thickness (mm)	-2.376243e-04
2	Coolant temp (°C)	-2.508596e-05
8	Mold temp (°C)	8.501532e-06
3	Injection pressure (bar)	7.525788e-07

```
import joblib
```

```
# STEP 7.1: Save the best model (Lasso Regression in this case)
joblib.dump(best_model, "cooling_time_model.joblib")
print("✅ Model saved as 'cooling_time_model.joblib'")
```

```
# STEP 7.2: Load the saved model
loaded_model = joblib.load("cooling_time_model.joblib")
```

```
# STEP 7.3: Predict using the loaded model to confirm it's working
y_loaded_pred = loaded_model.predict(X_test)
```

```
# Print R² score to check the performance of the loaded model
print("📊 Loaded Model R² Score:", r2_score(y_test, y_loaded_pred))
```

🔗 ✅ Model saved as 'cooling_time_model.joblib'
 📊 Loaded Model R² Score: 0.9999999999999857

